

3 Documento di Specifica dei Requisiti Software

3.1 Prospettiva del prodotto

BugBoard26 si colloca nel contesto degli strumenti di supporto allo sviluppo software, fornendo una piattaforma unificata per la gestione delle *issue*. Il sistema è concepito come un'applicazione indipendente, accessibile tramite interfaccia *web-based*, ma potenzialmente integrabile con altri strumenti di sviluppo.

Il prodotto adotta un'architettura client-server, in cui il client (interfaccia utente) comunica con un server centralizzato per la gestione dei dati relativi a utenti, issue, commenti e cronologie. Tutti i dati vengono conservati in un database relazionale, garantendo consistenza e tracciabilità.

3.2 Funzionalità del sistema

Il sistema offrirà le seguenti funzionalità principali:

- autenticazione sicura tramite email e password;
- gestione delle utenze da parte degli amministratori, con possibile aggiunta di scadenze opzionali per la risoluzione di bug;
- creazione delle issue con titolo, descrizione, tipologia, priorità e allegati.
- vista riepilogativa con possibilità di filtrare, ordinare e ricercare le issue secondo criteri multipli;
- assegnazione dei bug ai membri del team da parte degli amministratori, con relativo sistema di notifiche e area di visualizzazione personale;
- modifica dei bug nel rispetto del proprio livello di accesso.
- tracciamento della cronologia delle modifiche dei bug;

Queste funzionalità, qui scritte in linguaggio naturale, sono riscontrabili anche nello [Use Case UML](#) a pagina seguente.

3.3 Caratteristiche degli utenti

Gli utenti del sistema sono profilati secondo un modello di controllo degli accessi basato sui ruoli (RBAC - Role-Based Access Control), distinguendo due categorie gerarchiche:

Utente (Membro del Team): rappresenta l'attore base autenticato all'interno del sistema. Possiede i diritti di *lettura globale* che gli consentono di consultare l'elenco di tutte le segnalazioni (con facoltà di filtraggio e ordinamento) e di esaminarne il relativo storico delle modifiche dei bug. In *scrittura*, è abilitato ad aprire nuove segnalazioni (bug, feature, documentation, question). Tuttavia, può modificare lo stato e i dettagli esclusivamente dei *bug* di cui risulta formalmente assegnatario.

Amministratore (Admin): rappresenta un'utenza con privilegi di gestione (*superuser*) che estende le capacità dell'Utente base. Possiede la responsabilità esclusiva di registrare nuove utenze, assegnare i bug ai vari membri del team per la risoluzione e definire le relative scadenze. A livello di permessi, l'Amministratore non è soggetto al vincolo di assegnazione, godendo di diritti di modifica completi e incondizionati su qualsiasi issue o bug presente a sistema.

3.4 Use Case UML

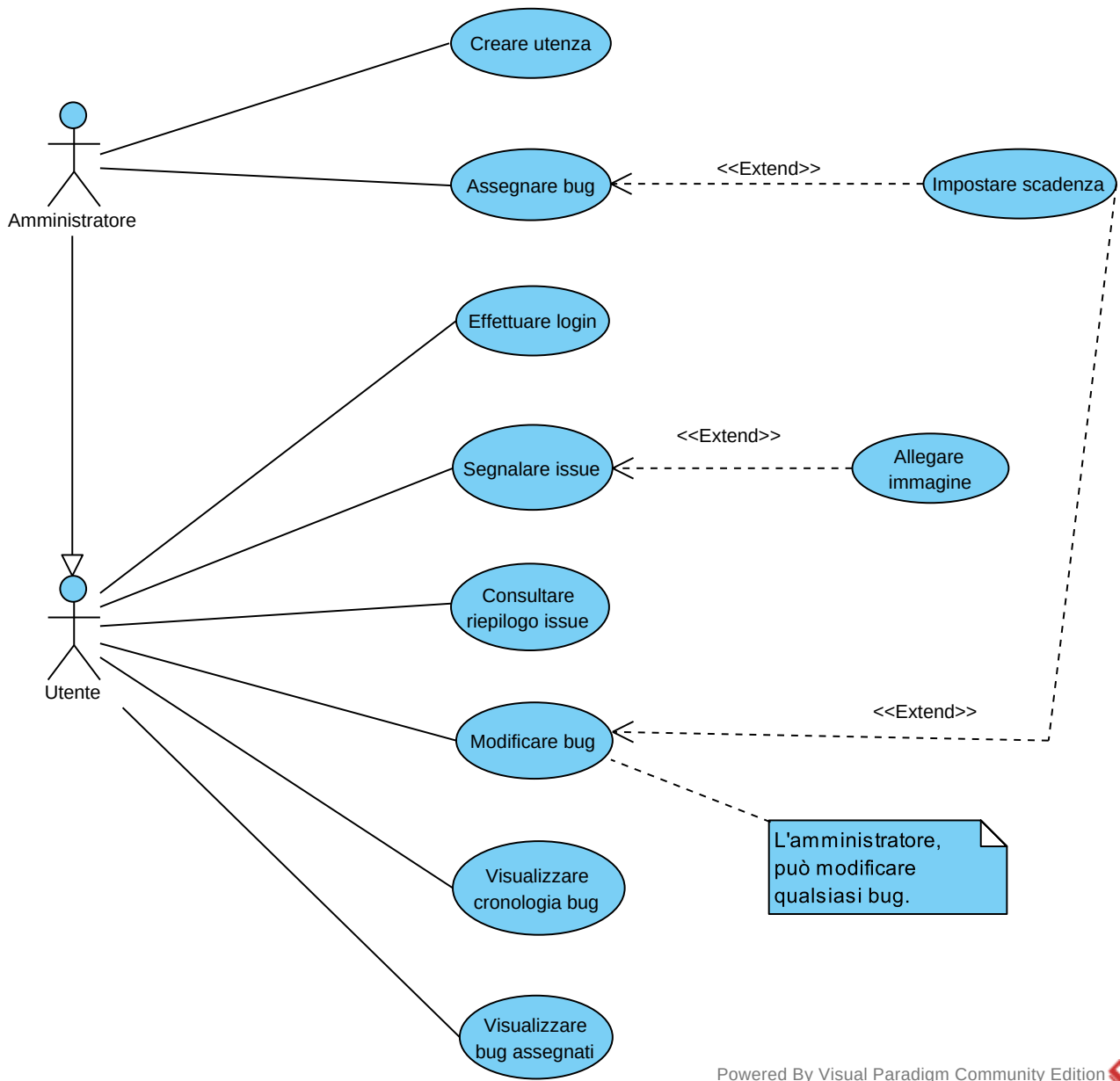


Figura 1: Use Case UML

3.4.1 Descrizione testuale strutturata

USE CASE #1	Segnalare issue		
Goal in context	Segnalare agli altri membri del team una issue su un progetto		
Preconditions	L'utente deve aver effettuato il login		
Success End Conditions	Il sistema tiene traccia della segnalazione della issue, che viene creata con stato iniziale "TODO"		
Failure End Conditions	La issue non viene aggiunta		
Primary Actor	Utente		
Trigger	Lo user clicca l'apposito bottone per segnalare issue		
Main Scenario	Step n.	Utente	System
	1	Clicca <i>Segnala issue</i>	
	2		Mostra M02
	3	Scrivo <i>Titolo</i> , <i>Descrizione</i> , inserisce il <i>Progetto</i> e specifica <i>Tipo</i>	
	4	Clicca <i>Ok</i>	
	5		Registra la issue e mostra M03
	6		Mostra M01
Extension #1 (clicca <i>Canc</i> e poi <i>Annulla</i>)	4a	Clicca <i>Canc</i>	
	5a		Mostra M04
	6a	Clicca <i>Annulla</i>	
	7a		Ritorna allo step 2 del main scenario
Extension #2 (clicca <i>Canc</i> e poi <i>Procedi</i>)	4b	Clicca <i>Canc</i>	
	5b		Mostra M04
	6b	Clicca <i>Procedi</i>	
	7b		Ritorna allo step 6 del main scenario
Extension #3 (specifica anche altre info)	3d	Scrivo <i>Titolo</i> e <i>Descrizione</i> , poi specifica <i>Progetto</i> , <i>Tipo</i> , priorità e allega immagini	
	4d	Clicca <i>Ok</i>	
	5d		Ritorna allo step 5 del main scenario

USE CASE #2	Assegnare bug		
Goal in context	Assegnare un bug a un membro del team per la risoluzione e, opzionalmente, fissare una scadenza		
Preconditions	L'Amministratore deve aver effettuato il login. Deve esistere almeno un bug nel sistema.		
Success End Conditions	Il bug è assegnato all'utente scelto, il quale riceve una notifica. L'eventuale scadenza viene registrata.		
Failure End Conditions	L'assegnazione non viene completata. Il bug rimane nel suo stato precedente.		
Primary Actor	Amministratore		
Trigger	L'amministratore clicca sul bottone per assegnare uno specifico bug		
Main Scenario	Step n.	Admin	System
	1	Clicca <i>Assegna Bug</i>	
	2		Mostra M05
	3	Seleziona <i>Progetto, Bug</i> ed <i>Utente</i> dalla lista	
	4	Clicca <i>Ok</i>	
	5		Mostra M07 ed invia notifica all' <i>Utente</i>
	6		Mostra M01
Extension #1 (<i>Admin</i> sceglie di impostare scadenza)	3a	Inserisce data in <i>Scadenza</i>	
	4a		Ritorna allo step 5 del main scenario
Extension #2 (clicca <i>Canc</i> e poi <i>Procedi</i>)	4b	Clicca <i>Canc</i>	
	5b		Mostra M06
	6b	Clicca <i>Procedi</i>	
	7b		Ritorna allo step 6 del main scenario
Extension #3 (clicca <i>Canc</i> e poi <i>Annulla</i>)	4d	Clicca <i>Canc</i>	
	5d		Mostra M06
	6d	Clicca <i>Annulla</i>	
	7d		Ritorna allo step 2 del main scenario

3.4.2 Mock-Ups

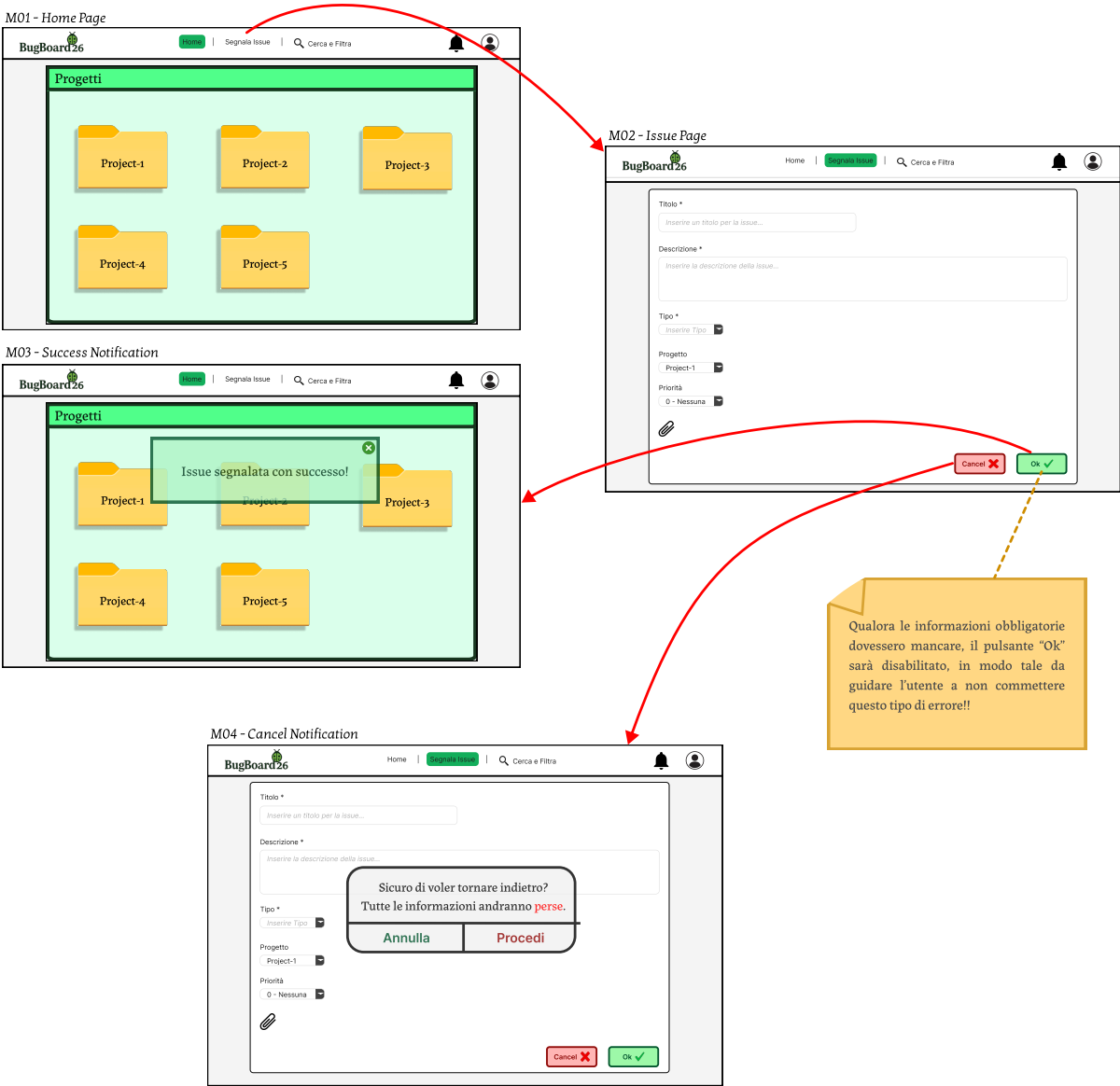


Figura 2: M01-M04

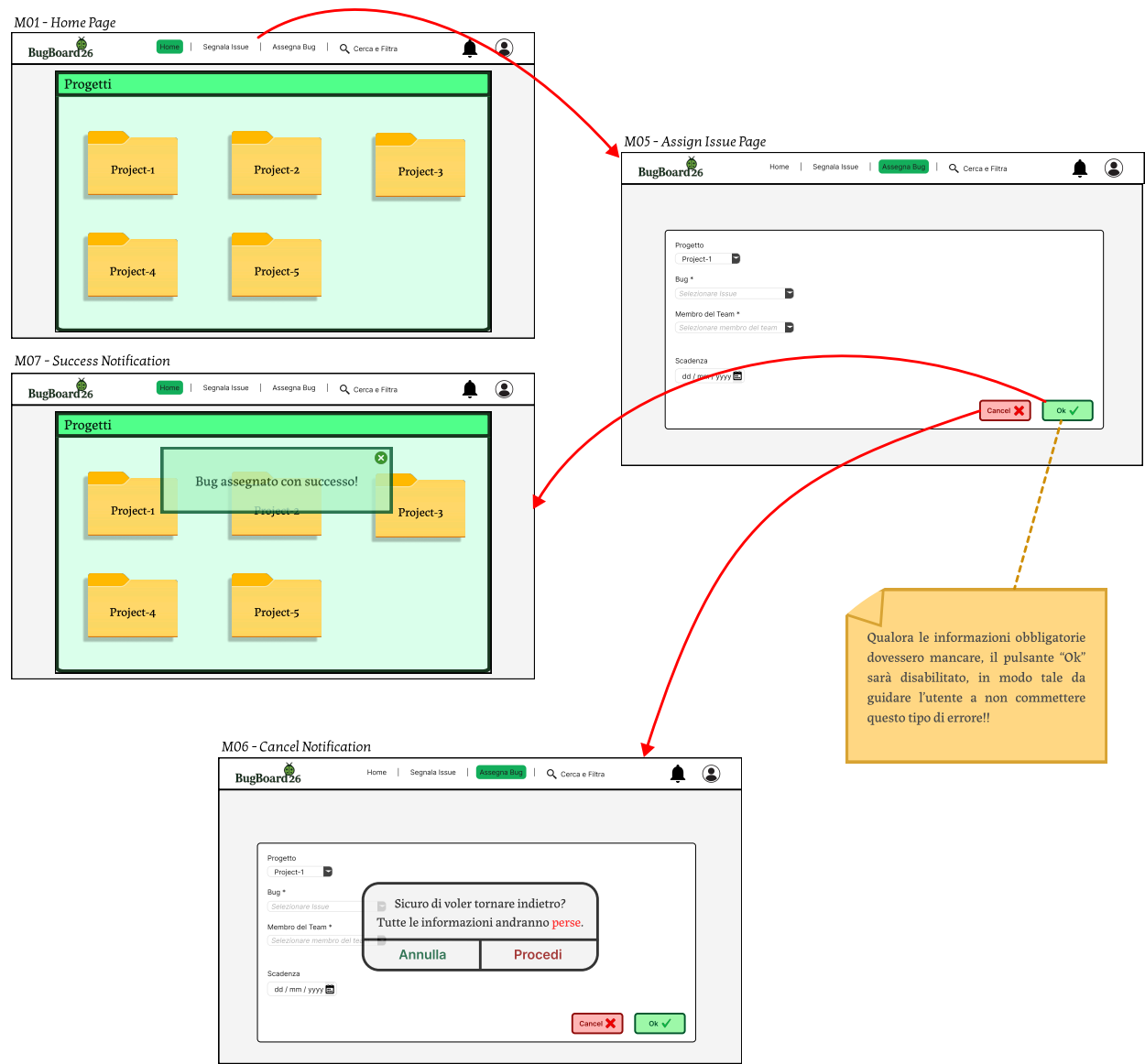


Figura 3: M01, M05-M07

3.5 Modello del Dominio del Problema

Al fine di formalizzare le entità del dominio e le loro relazioni concettuali, viene introdotto il *Modello del Dominio del Problema*.

A differenza dei diagrammi delle classi di progettazione, questo modello astrae totalmente dai dettagli tecnologici e implementativi. Si concentra unicamente sui concetti del mondo reale e sui vincoli di business.

Per gestire la complessità e garantire la massima chiarezza espositiva, la modellazione del dominio verrà presentata in maniera modulare e strutturata per Use Case: per ciascun caso d'uso verrà mostrato e descritto il relativo diagramma concettuale delle entità partecipanti.

3.5.1 Segnalare Issue

In questa prima vista concettuale vengono modellate le entità dal punto di vista dell'attore *Utente*, coerentemente con l'esecuzione dello Use Case *Segnalare Issue*.

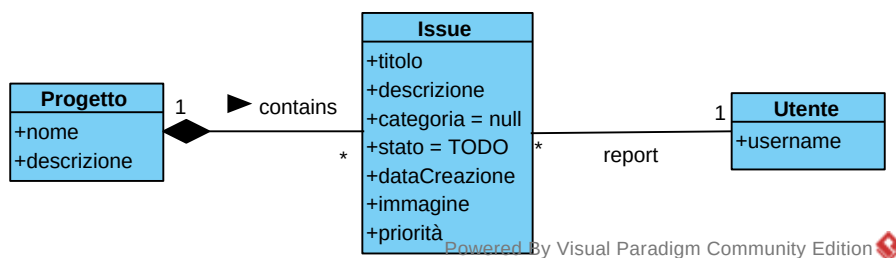


Figura 4: Segnalare Issue

3.5.2 Assegnare Bug

In questa vista vengono modellate le entità dal punto di vista dell'attore *Amministratore*, in accordo con lo Use Case *Assegnare Bug*.

Qui si evidenzia l'estensione dei privilegi dell'Amministratore rispetto all'Utente base. L'Amministratore interagisce con l'entità **Bug** (specializzazione concettuale della generica **Issue**), con responsabilità esclusiva di assegnarne la risoluzione a un membro del team (**Utente**) e di definirne una **scadenza**, qualora necessaria.

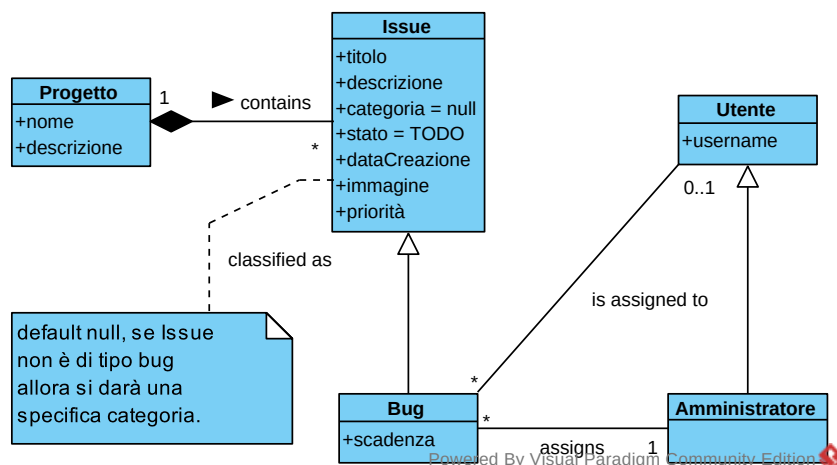


Figura 5: Assegnare Bug

3.5.3 Creare Utenza

In questa vista viene modellata l'interazione concettuale per la gestione e la creazione delle utenze all'interno della piattaforma, in conformità con lo Use Case *Creare Utenza*.

L'attore **Amministratore** detiene la responsabilità esclusiva di registrare nuovi utenti all'interno del sistema, definendo le credenziali identificative dell'account e assegnando il relativo livello di privilegio (ruolo di **Utente** base o di **Amministratore**).

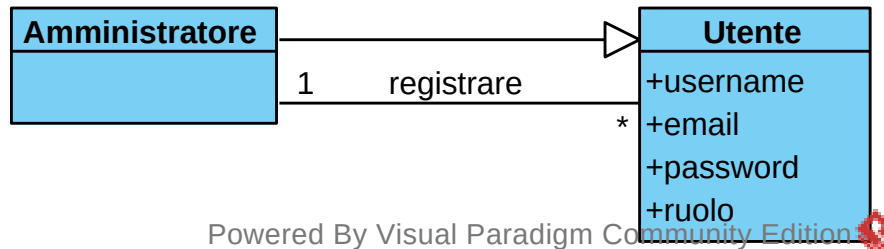


Figura 6: Creare Utenza

3.5.4 Effettuare Login

In questa vista viene modellato il processo di verifica dell'identità per l'accesso alla piattaforma, corrispondente allo Use Case *Effettuare Login*.

Prima di poter operare sul sistema, l'**Utente** deve autenticarsi con una e-mail e password. Il superamento di tale verifica si traduce, a livello concettuale, nella creazione di una **SessioneAutenticata**. Certificato in questo modo, l'Utente potrà ora accedere a tutte le funzionalità offerte.

Per garantire la sicurezza applicativa e la protezione dei dati, l'entità prevede l'attributo booleano *scaduta* con valore di default *false*.

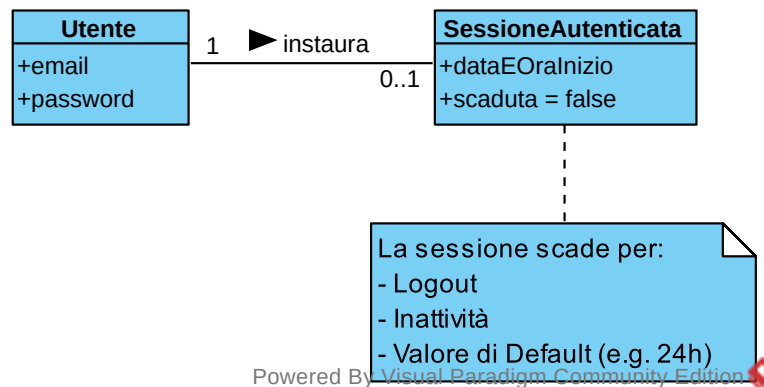


Figura 7: Effettuare Login

3.5.5 Consultare Riepilogo Issue

In questa vista concettuale viene modellata l'interazione relativa alla consultazione, ricerca, filtraggio e ordinamento delle segnalazioni, in conformità con lo Use Case *Consultare Riepilogo Issue*.

Per caratterizzare dinamicamente questa interazione di interrogazione senza inquinare le entità di dominio principali, è stata introdotta la classe associativa **Ricerca**. Essa incapsula parametri di filtraggio puramente opzionali. Qualora ciò non siano sufficienti, *testoLibero* permette all'utente di ricercare per *titolo* e *descrizione* di Issue. Infine, ha la facoltà di scegliere l'ordinamento di filtraggio (*ASC*, *DESC*...)

In assenza di criteri, l'attributo **ordinamento** definisce il comportamento di default del sistema, presentando i risultati ordinati storicamente in base alla data di creazione crescente (*CREATED_ASC*).

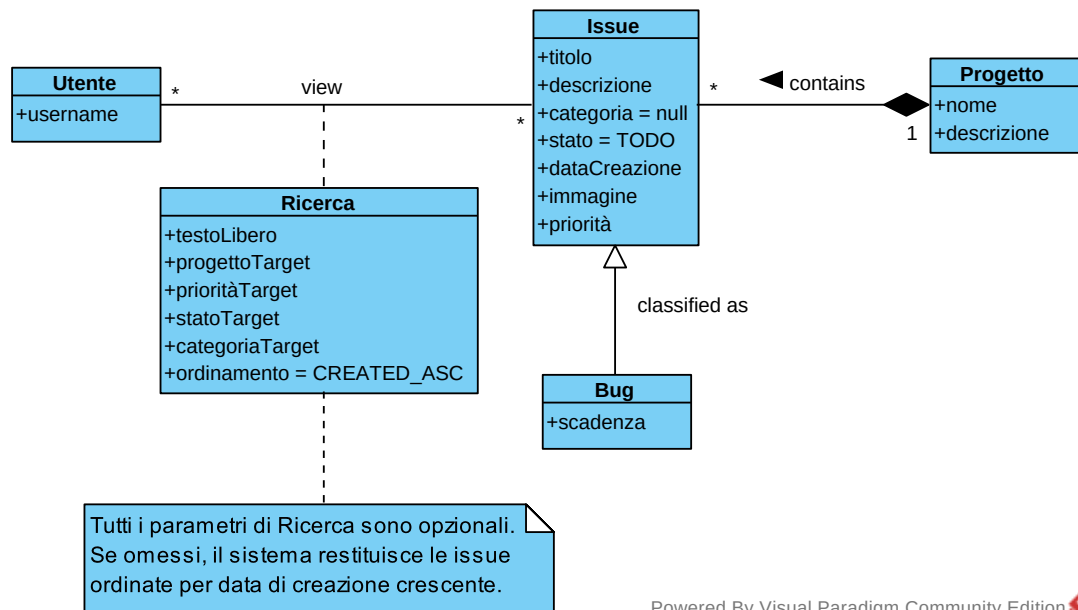


Figura 8: Consultare Riepilogo Issue

3.5.6 Visualizzare Bug Assegnati

In questa vista concettuale viene modellata la prospettiva dell'attore **Utente** in merito alla visualizzazione dei bug assegnati.

L'**Utente** visualizza i vari **Bug** di cui è responsabile ed è caratterizzata dalla classe associativa **Notifica**, che avviene nel momento dell'assegnazione da parte dell'**Amministratore**. L'entità **Notifica** incapsula il messaggio di avviso, il timestamp di generazione (**dataEOraInvio**) e lo stato di lettura (**letta**), consentendo all'utente di monitorare in modo puntuale i task assegnati e le relative scadenze.

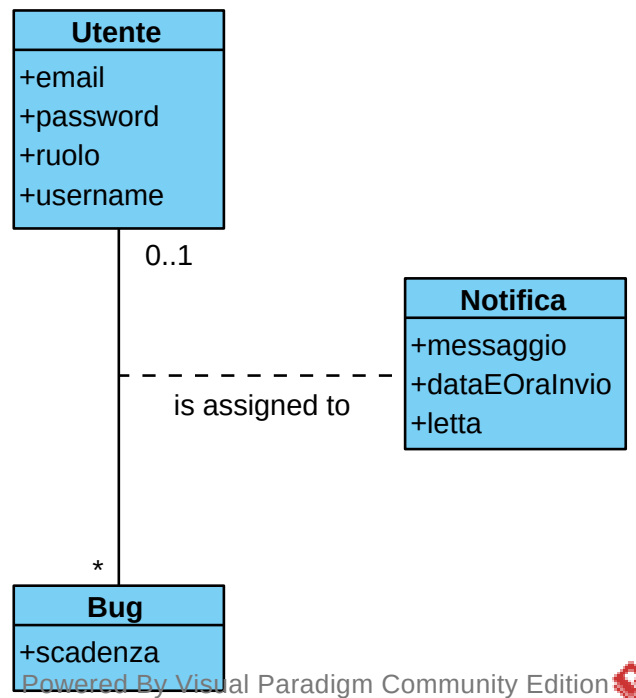


Figura 9: Visualizzare Bug Assegnati

3.5.7 Visualizzare Cronologia Bug

In questa vista concettuale viene modellata la prospettiva dell'attore **Utente** relativa alla consultazione del tracciamento delle modifiche, in conformità allo Use Case *Visualizzare Cronologia Bug*.

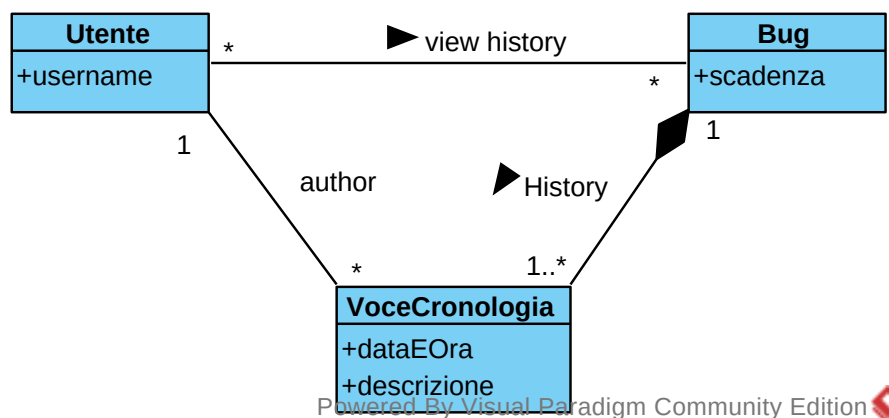


Figura 10: Visualizzare Cronologia Bug

3.5.8 Modificare Bug

In questa vista concettuale viene modellata la gestione della modifica delle informazioni di un bug e le relative regole di autorizzazione e controllo accessi, in conformità con lo Use Case *Modificare Bug*.

Il modello formalizza la differenziazione dei privilegi operativi:

- **Utente:** Possiede la facoltà di apportare modifiche (aggiornamento di stato, priorità, descrizione...) esclusivamente ai **Bug** di cui risulta formalmente assegnatario. La relazione ha molteplicità $0..1 \leftrightarrow *$: un bug può non essere assegnato ad alcun utente (0) o essere assegnato a un unico utente (1), il quale può modificare tutte le segnalazioni a lui attribuite (*);
- **Amministratore:** In qualità di attore con privilegi elevati (**Amministratore** $\succ$ **Utente**), dispone di accesso completo su qualsiasi **Bug**. La relazione ha molteplicità multi-a-molti ($* \leftrightarrow *$): ciascun amministratore può modificare molteplici bug e ogni bug è modificabile da qualsiasi amministratore presente nel sistema.

La vista mantiene soppressa la superclasse **Issue** per garantire la massima essenzialità visiva, concentrando l'attenzione sugli attributi modificabili e sui permessi degli attori.

Si è deciso di offrire una doppia vista per facilitare la visione della differenza nell'utilizzo dello Use Case per ogni attore, ma nella realtà, l'**Amministratore** potrà modificare qualsiasi parametro di Bug.

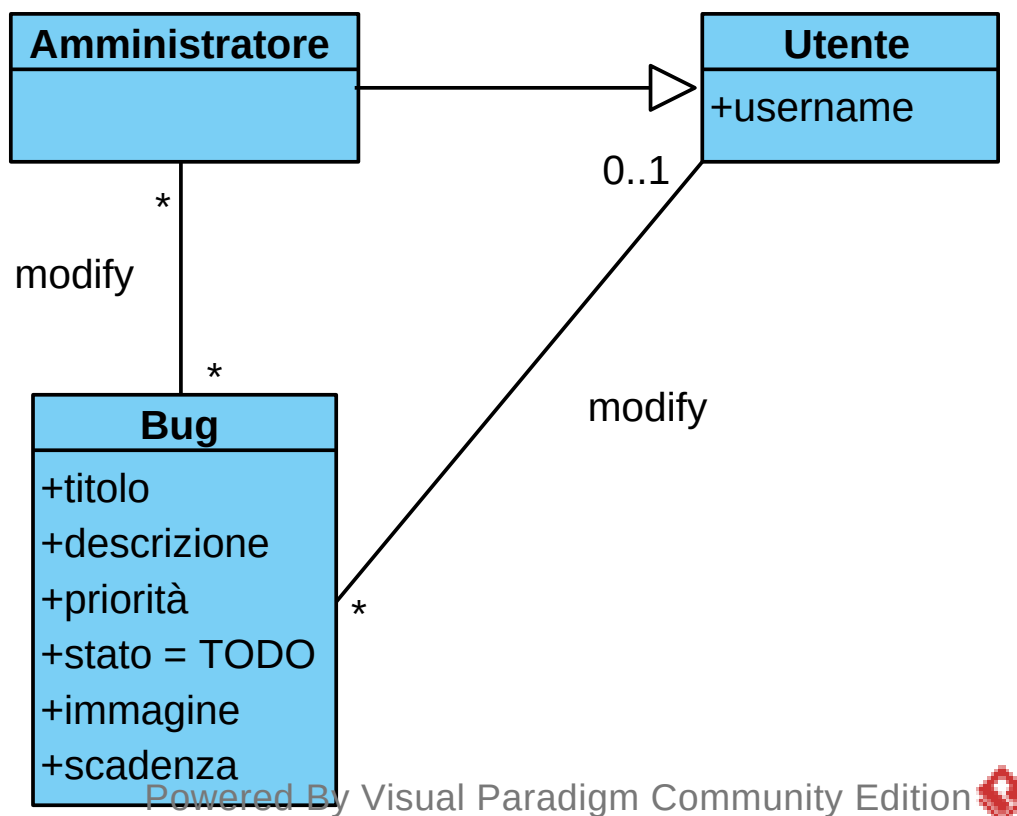


Figura 11: Modificare Bug

3.6 Personas

Al fine di identificare più chiaramente il target utenti di cui si è parlato poc'anzi, si è lavorato con lo strumento dei *Personas*. Nelle pagine seguenti sono riportati due esempi, uno per ogni tipologia di utente.

3.6.1 Persona 1

Roberto Esposito

BIO

Con un approccio lungimirante e determinato, Roberto Esposito lavora come Project Manager presso la KoopaTech, in ambito IT. Laureato all'Università degli Studi di Napoli Federico II, ha intrapreso un percorso professionale che lo ha portato a gestire team e progetti in contesti dinamici e sfidanti. Ora è un leader competente e attento ai dettagli, alla ricerca di nuove tecnologie per rendere più competitiva la sua azienda. Fuori dall'ufficio, è una persona fortemente legata alla famiglia: Roberto, infatti, è felicemente sposato ed è padre di due gemelle, con cui condivide la passione per il cinema.

"Efficienza è la strategia, innovazione è lo strumento."

About

- 48
- Federico II, Napoli
- Napoli, Italia
- Maschio
- Project Manager
- Sposato

Lungimirante Smargiasso Competitivo

FAVORITE BRANDS

- Red Bull
- GitHub
- Udacity
- Napoli FC

GOALS

- Gestire diversi progetti in contemporanea in modo chiaro.
- Mantenere il controllo sugli account e sui permessi utente.
- Migliorare le performance aziendali.

NEEDS

- Avere una visione complessiva dello stato dei progetti e delle issue aperte.
- Permettere, in maniera sicura, di mostrare i progetti agli stakeholder.
- Monitorare le performance aziendali.

INTERESTS

- Sport: gli piace organizzare partite di calcio con i suoi vecchi compagni.
- Cinema: trova affascinanti i film dell'orrore e i racconti d'autore.
- Famiglia: ritiene importante curare i rapporti all'interno della sua famiglia.

PAIN POINTS

- Mancanza di tracciabilità nelle modifiche effettuate dagli sviluppatori.
- Tempi di attesa proibitivi per operazioni semplici.
- Piattaforme con una curva di apprendimento troppo ripida.

Figura 12: Roberto Esposito, Admin

3.6.2 Persona 2



*"Il codice è classe,
ogni bug è una lezione."*

About

- 22
- Bocconi, Milano
- Milano, Italia
- Maschio
- Junior Developer
- Single
- Motivato
- Impulsivo
- Empatico

FAVORITE BRANDS



Francesco Sarto

BIO

Francesco Sarto è un giovane sviluppatore appena uscito dall'università con una laurea in Informatica. Appassionato di tecnologia fin da ragazzo, ha iniziato a programmare a 14 anni, creando piccoli progetti personali. Attualmente lavora come junior developer in una startup dove si occupa principalmente di sviluppo back-end con C++ e Java. È sempre alla ricerca di nuove sfide per migliorare le sue competenze e ama partecipare a hackathon e meetup locali. Preferisce lavorare in team e valorizza il feedback dei colleghi più esperti. Nel tempo libero contribuisce a progetti open source, ma è anche un appassionato di videogiochi, tant'è che frequenta fiere a tema, come il Milan Games Week.

GOALS

- Collaborare con il team tramite commenti.
- Monitorare lo stato delle issue assegnate in modo chiaro.
- Contribuire attivamente a progetti.

INTERESTS

- Gaming: appassionato di videogiochi platform e rogue-like.
- UI/UX design: non esperto, ma molto curioso in materia.
- Networking: partecipa regolarmente a meetup tech e conferenze.

NEEDS

- Disponibilità di chat integrata per fornire suggerimenti o richiedere chiarimenti.
- Ricevere notifiche sugli aggiornamenti delle issue assegnate.
- Interfaccia semplice e pronta all'uso.

PAIN POINTS

- Difficoltà nel filtrare e cercare le informazioni che ritiene rilevanti.
- Mancanza di shortcut per ottimizzare processi di routine.
- Strumenti che non garantiscono una comunicazione efficace con i Senior dev.

Figura 13: Francesco Sarto, Developer

3.7 Requisiti non funzionali e di dominio

1. **Accesso concorrente:** il sistema deve supportare l'accesso concorrente da parte di almeno 20 utenti senza degrado significativo delle prestazioni.
2. **Architettura distribuita:** il sistema deve seguire le regole di un *sistema distribuito*, comprendente:
 - utilizzo del multithreading;
 - indipendenza tra back-end e front-end;
 - utilizzo di un database per garantire la persistenza dei dati;
 - accessibilità tramite rete Internet;
 - tolleranza ai guasti.
3. **Best practice:** il codice dovrà seguire standard di buona pratica (es. naming convention, documentazione inline, testing unitario);
4. **Conformità al GDPR:** il sistema deve rispettare i principi stabiliti dal *Regolamento (UE) 2016/679*, garantendo la tutela dei dati personali degli utenti. In particolare:
 - raccolta e trattamento dei dati personali solo previo consenso esplicito dell'utente;
 - diritto alla modifica e cancellazione dei propri dati;
 - protezione dei dati tramite tecniche di cifratura e controllo degli accessi;
 - conservazione dei dati limitata al tempo strettamente necessario per le finalità dichiarate.
5. **Programmazione ad oggetti:** il sistema deve essere sviluppato utilizzando un linguaggio OO.
6. **Riservatezza:** il sistema deve garantire la sicurezza e la riservatezza delle informazioni mediante meccanismi di autenticazione e autorizzazione.
7. **Astrazione e Riuso:** in fase di progettazione è richiesto tassativamente di astrarre il design per favorire il riutilizzo del codice e la futura implementazione di altre funzionalità.
8. **Sicurezza applicativa:** il sistema deve essere protetto contro i principali attacchi informatici, come *Cross-Site Scripting (XSS)*, *Cross-Site Request Forgery (CSRF)* e *SQL Injection*, adottando linee guida e controlli previsti dagli standard OWASP.
9. **Sicurezza delle comunicazioni:** tutte le comunicazioni tra client e server devono avvenire tramite API REST, cifrando le informazioni sensibili.
10. **Standard ISO/IEC 25010:** il sistema deve rispettare le linee guida definite dallo standard per la qualità del software.
11. **Tecnologie:** il sistema deve essere sviluppato con tecnologie standard e manutenibili, con la possibilità di effettuare test unitari.
12. **Usabilità e accessibilità:** l'interfaccia utente deve essere intuitiva e reattiva. Le principali operazioni (creazione di issue, commento, filtraggio) devono poter essere eseguite in un numero massimo di tre interazioni da parte dell'utente.

3.8 Assunzioni e dipendenze

Lo sviluppo e il corretto funzionamento del sistema dipendono dalle seguenti condizioni:

- disponibilità di una connessione di rete stabile per la comunicazione tra Front-end e Back-end.
- presenza di un server per la persistenza dei dati gestito dal Back-end.
- utilizzo obbligatorio di tool di CASE (es. Visual Paradigm) per la stesura della progettazione;
- utilizzo di un sistema di versioning (es. repository GitHub) per tracciare lo sviluppo e gli artefatti prodotti.
- utilizzo di un sistema operativo compatibile con la piattaforma scelta;
- eventuale integrazione con servizi esterni (es. email o notifiche push) avverrà tramite API documentate e sicure.